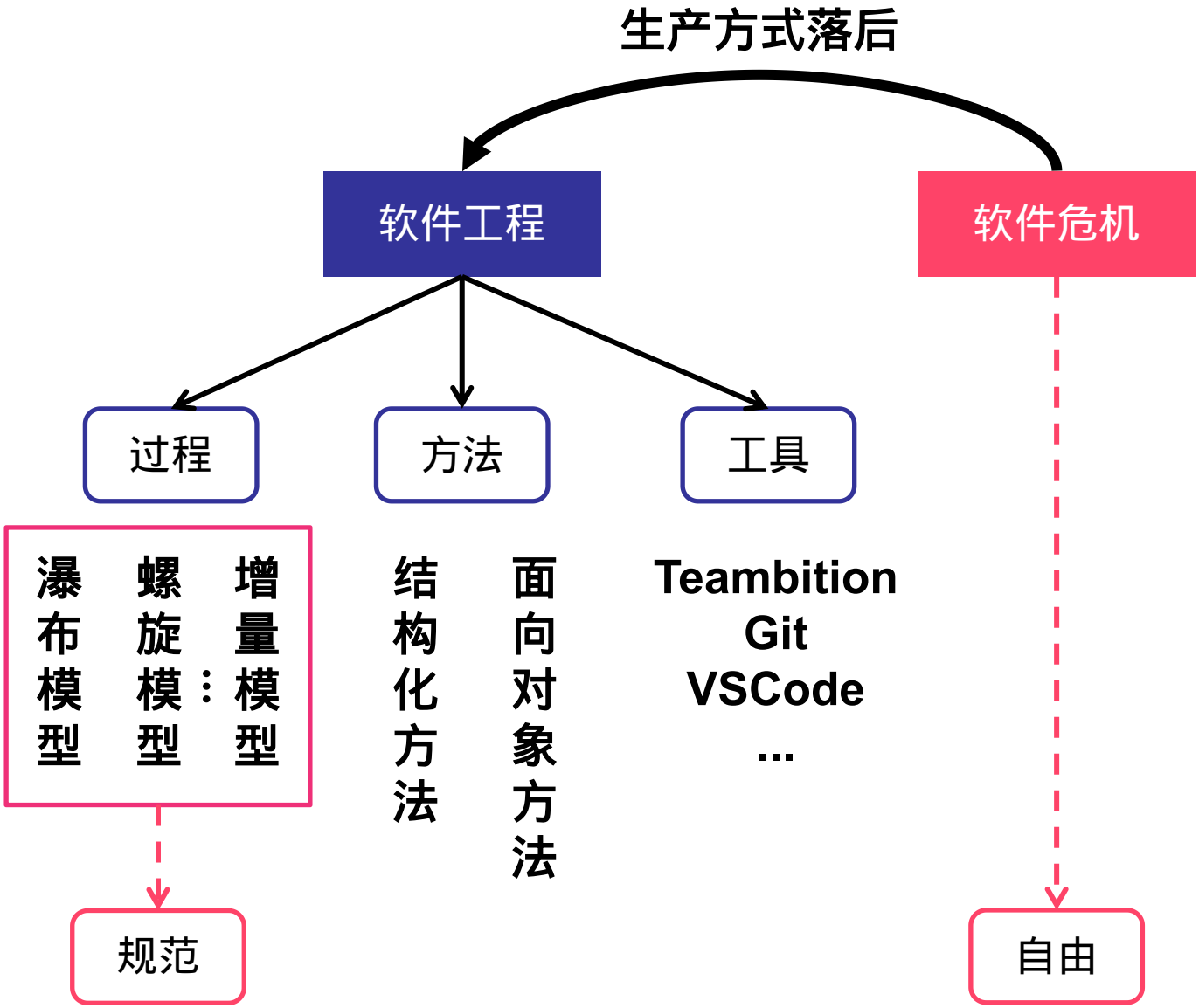
第二节 敏捷开发与项目协作

肖逸飞
xyf1989@uestc.edu.cn

课程安排

1. 软件工程与软件建模
2. 敏捷开发与项目协作
3. 开发环境规范与测试
4. AI辅助开发
5. 需求设计与程序框架
6. 打包解包及其关键技术
7. 压缩解压及其关键技术
8. 加密解密与系统集成
9. 软件发布与项目收尾
10. 项目答辩与成果展示

传统软件开发过程的问题



传统软件过程的问题

软件开发实质为一种**手工过程**。为保证项目成功，传统的软件过程模型均对软件开发过程进行控制，但是这种控制其实是一种**过度规范**（如能力成熟度模型CMMI），主要表现：

1. 严格的流程管理；
2. 复杂的管理和开发工具；
3. 详尽的文档；
4. 和甲方的合同谈判；
5. 严格的执行计划；

这些都是阻碍软件开发高效执行的因素，于是……



敏捷开发宣言

2001年，17位工程师，《敏捷软件开发宣言》

敏捷开发宣言

2001年，17位工程师，《敏捷软件开发宣言》

我们一直在实践中探寻更好的软件开发方法，身体力行的同时也帮助他人。由此我们建立了如下4个价值观:

响应变化 > 遵循计划

工作的软件 > 详尽的文档

客户合作 > 合同谈判

个体和互动 > 流程和工具

也就是说，尽管右项有其价值，我们更重视左项的价值。

敏捷开发的核心思想：

把大的完整软件分成一个个小的功能(story)，每个功能用较短的周期进行开发，并短周期经常性地交付可工作的软件。

敏捷开发工具/流派

1、看板

2、Scrum

3、极限编程（XP）

看板

用不同颜色的小纸片上注明任务的各种属性。用列，每一列代表任务的一个阶段，





看板

其他概念

Cycle Time：一张代表任务的小纸片在白板上，从左到右，经历所有流程所花费的时间（越小越好）。
Throughput：在单位时间内，在白板上经历所有流程，完成任务的小纸片的数量，可以根据每个任务的大小加权计算（越高越好）。
Swimlane（泳道）：从另一个维度增强看板的功能（当纸片太多）。

状态列 (横轴) ➔	待办 (Todo)	开发中 (Doing)	已完成 (Done)
泳道 1: 【加急特快线】	[卡片] 线上崩溃Bug	[卡片] 支付网关报错	
泳道 2: 【常规功能线】	[卡片] 完善用户中心 [卡片] 积分抵扣开发	[卡片] 导出PDF功能	[卡片] 登录接口
泳道 3: 【技术债/基础设施】		[卡片] 数据库调优	[卡片] 清理旧日志

看板

最佳实践

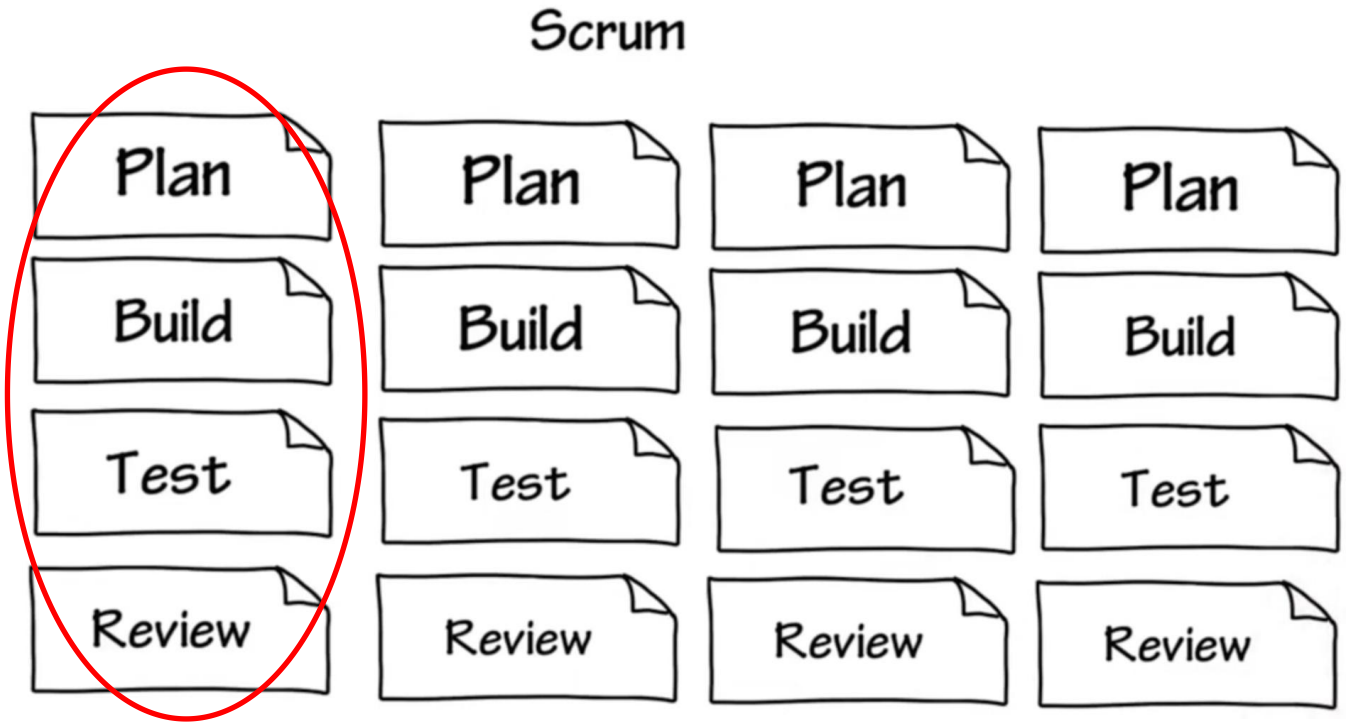
1、 将看板摆在项目组中最显眼的位置。
2、 为团队每日站会提供讨论依据。
3、 可以方便的和其它敏捷开发方法做集成。



Scrum

最常用的敏捷开发方法之一。
核心思想：小步快跑，逐渐迭代。

Sprint：Scrum把软件开发的过程从时间上分为不同的Sprint（冲刺），每个Sprint的时间可以是1到4周，通常是用2周，一个Sprint包含一个Sprint周期中需要完成的用户功能（user storys）。



Scrum-三种角色



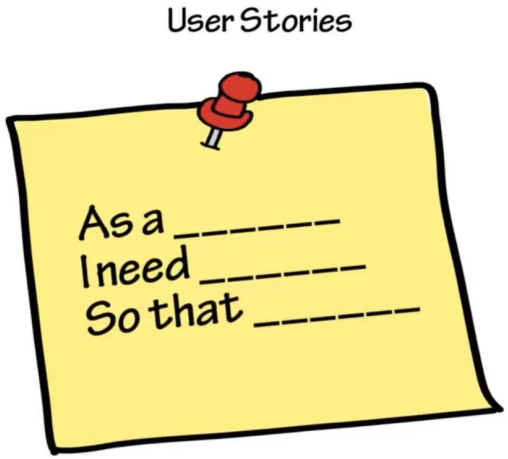




产品经理（PO）	Scrum教练（SM）	开发团队
做什么 为什么做 做出来东西对不对	负责如何正确的做	怎么做

Scrum-三种文档

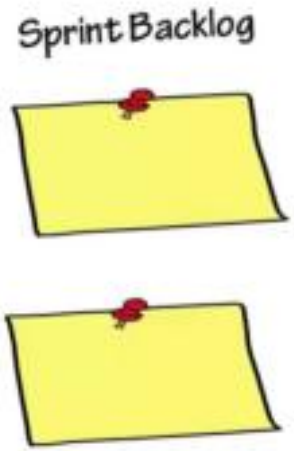
用户故事是以这样的三段式写作的，描述的是一个软件功能：

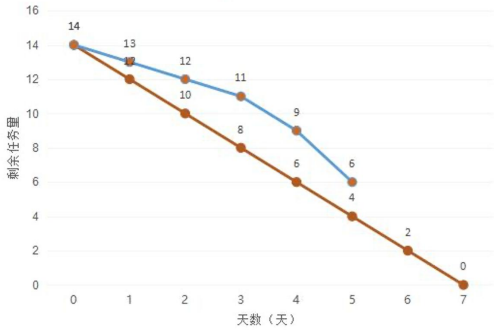


比如：“作为一个用户，我需要能够登录系统，以便访问我的个人资料。”

产品代办列表	Sprint代办列表	燃尽图
用户故事、调研工作等	用户故事以及相关任务	描述团队开发效率







Scrum-四种会议

四种会议：

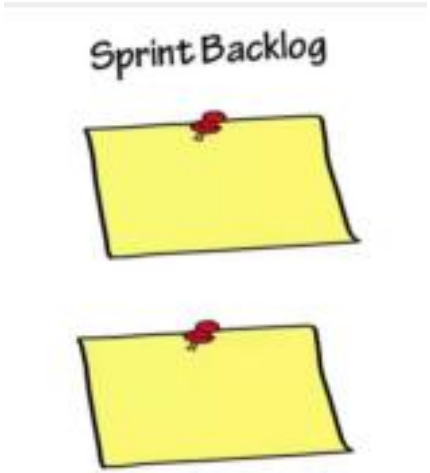
1. Sprint计划会
2. 每日站会
3. Sprint评审会
4. Sprint回顾会

Sprint计划会

参与人员：所有Scrum成员（**SM、 Team、 PO**）
时间：1周*2h



根据优先级进行筛选



每日站会

参与人员：SM、Team、PO（可选）
时间：15min

每个团队成员轮流回答三个问题：

> 我昨天做了什么来帮助团队达成Sprint目标？
> 今天我打算做什么来帮助团队达成Sprint目标？
> 我遇到了什么障碍（Impediment）？

Sprint评审会

参与人员：Scrum团队、利益相关方（客户、用户、管理层等）
时间：1周*2h

检阅本Sprint的产出成果，并根据反馈调整未来的方向。

Sprint回顾会

参与人员：Scrum团队
时间：1周*45min

团队讨论上个Sprint中哪些方面**做得好**，应该继续保持。

团队讨论哪些方面**可以改进**。

团队共同制定出一项或几项具体的、可在下个Sprint中实施的**改进措施**。

Scrum Master会协助团队确保回顾会富有成效，并**跟进**改进措施的落实。

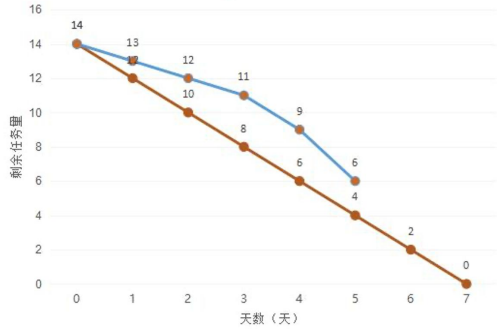
Scrum核心概念

三种角色：
Product Owner
Team
Scrum Master

三种工具：
User Story
Product/Sprint Backlog
Burndown Chart

四种会议：
Sprint计划会
每日站会
Sprint评审会
Sprint回顾会





大牛对Scrum的介绍：https://www.bilibili.com/video/BV1rW411K7vD

Scrum注意事项

角色人选：
Product Owner
Scrum Master

技术还债：定期重构
Scrum反例：

站会内容（变成了“日报流水账汇报”、“陷入深度技术细节”、“虚无日常”）
PO和SM合一
越级管理（管理者对开发团队过度干涉、绝对控制、剥夺团队自组织能力）

死法一：绝对的任务指派（强行分工）

死法二：工时和过程的“变态级监控”

死法三：扼杀技术决策，搞“一言堂”

敏捷开发示例

Sprint 1：核心骨架与本地打包（目标：拿到单机备份核心分）

本周目标：实现最核心的“打包-解包”链条，确保软件能把文件抓进来、合在一起、再还原回去。

用户故事 (User Story)：作为用户，我能把一个文件夹打包成一个备份文件，并能完整解包还原。

本周冲刺任务（Sprint Backlog）：

打包解包（10分）：设计自定义的打包大文件格式（Tar格式或自定义二进制流），实现拼接与拆解。

文件类型与元数据基础（10+10分）：优先支持普通文件和目录，在打包时顺便把权限（chmod）、时间戳（mtime）写进大文件的头部。

图形界面基础（10分）：用 Qt 或 Electron 勾勒出一个极简界面，只有“选择源目录”、“选择备份路径”、“开始备份/还原”三个按钮。

周五交付物：一个有丑陋界面的单机软件，能点按钮把本地文件夹打包成一个 .bak 文件，并能成功还原。

敏捷开发示例

Sprint 2：单机功能增强（目标：把单机策略和安全分数拿满）

本周目标：在第一周稳定的骨架上，把压缩、加密和筛选功能像“插头”一样接进去。

用户故事：作为用户，我希望备份文件更小、更安全，且能过滤掉不需要的垃圾文件。

本周冲刺任务（Sprint Backlog）：

压缩解压（10分）：引入一种算法（如哈夫曼或 Zstd），在打包后/打包时进行数据压缩。

加密解密（10分）：引入一种算法（如 AES-256），用户在 GUI 输入密码，对压缩后的数据加密。

自定义备份（3分×6=18分）：在 GUI 增加筛选配置面板。在扫描本地文件时，加入对路径、类型、名字、时间、尺寸、用户的 Filter 过滤逻辑。

周五交付物：一个功能完善的单机版备份专家。用户可以通过界面勾选“只要 .txt 文件”，输入密码，生成一个加密且压缩的备份包。

敏捷开发示例

Sprint 3：自动化与高级文件支持（目标：攻克单机最高难度的实时备份）

本周目标：从“手动点按钮”进化为“自动、定时、实时”备份，并解决底层硬核文件系统的兼容性。

用户故事：作为用户，我不用管它，软件自己就能在后台帮我默默备份变化的文件。

本周冲刺任务（Sprint Backlog）：

高级文件与元数据（冲刺残余分数）：攻克特殊文件系统（软硬链接、管道、设备文件）的读写与元数据（属主等）的备份。

定时备份（10分）：引入 Cron 表达式或定时器线程，实现周期备份。设计“数据淘汰”策略（如只保留最近3个版本的备份，旧的删除）。

实时备份（15分 - 重头戏）：利用操作系统的底层通知机制（Windows 用 ReadDirectoryChangesW，Linux 用 inotify）。监控到文件创建/修改后，立刻触发备份逻辑。

周五交付物：完美的单机全功能版。不仅支持所有特殊文件，还能在后台静默运行。你在监控目录下新建一个文件，软件瞬间在后台完成过滤、打包、压缩、加密并淘汰旧数据。

敏捷开发示例

Sprint 4：网络版转型（目标：单机变网盘，攻克分布式基础）

本周目标：撕掉本地存储，把备份文件通过网络丢到服务器上，实现 C/S 架构。

用户故事：作为用户，我可以在任何电脑上登录我的账号，把文件备份到云端服务器。

本周冲刺任务（Sprint Backlog）：

网盘模式与用户管理（10+5分）：搭建 Server 端。实现客户端注册、登录、Token 验证。

元数据管理（5分）：服务器不仅存大文件，还要用数据库（如 SQLite/MySQL）记录哪个用户在什么时间上传了什么文件，方便检索。

传输加密（5分）：客户端与服务端通信采用 TLS/HTTPS，或者对传输的 Socket 数据流用前面的加密算法再套一层，防止网络抓包。

周五交付物：网络版原型。用户在客户端登录后，点击备份，数据不再存在本地，而是通过网络安全地传输到服务器存起来。

敏捷开发示例

Sprint 5：终极优化：增量备份与加分项

本周目标：解决网络备份的最大痛点——带宽。只传变化的部分，并打磨产品。

用户故事：作为网盘用户，我修改了一个 1GB 文件里的一个字，我希望几秒钟就能备份完，而不是重新传 1GB。

本周冲刺任务（Sprint Backlog）：

增量备份（5分 - 核心难点）：引入备份对比机制。可以是文件级增量（对比 mtime 或 MD5，变了才传），或者是更高阶的块级增量（类似 rsync 算法，只传文件内改变的 Data Block）。

多算法扩展（多拿分）：如果学有余力，在打包/压缩/加密处，各多堆叠一种算法（例如压缩支持 Gzip 和 Bzip2，加密支持 AES 和 SM4），每种算法多拿10分！

其它加分项尝试：断点续传、秒传（通过文件哈希直接在服务器匹配）、备份进度条与速度显示。

周五交付物：终极完整版。具备商业级网盘素质的增量备份软件，答辩演示时惊艳全场。

极限编程

另一个常用的敏捷开发方法之一。
核心思想：将目前公认的对编程有益的理念进一步强调并发挥到极致。

举例：

软件测试是重要的
代码审查是重要的
用户反馈是重要的

先写测试用例再写代码（TDD）
结对编程
要求客户直接参与到代码生产中

极限编程

极限编程的12条具体实践：

精细反馈：结对编程、计划博弈、测试驱动开发、现场客户

持续过程：持续集成、代码重构、小型发布

共享认知：编码标准、系统命名隐喻、代码集体共有、简单设计

程序员福利：可持续步调

极限编程

系统命名隐喻举例：

假设团队正在开发一个**电子商务平台**。

没有系统隐喻时，对话可能是这样的：

程序员A：“我在那个处理用户请求的Servlet里加了点逻辑，调用DAO层之前先检查一下Redis缓存里的KV值，如果命中就直接返回，不然就穿透到MySQL用ORM映射对象，再回写到缓存里。”

产品经理：“...哦。”（完全没听懂，无法参与讨论）

新入职的程序员C：“...我得先看看你的代码是哪个类的。”（理解成本高）

这种沟通充满了技术黑话（Servlet, DAO, Redis, KV, ORM），非技术人员和新人根本无法快速理解。

- **数据库MySQL = 中央大仓库**：所有商品信息、用户订单最终都安全地存放在这里。
- **Redis缓存 = 仓库门口的临时货架**：存放最热门、最畅销的商品，方便快速取用，避免每次都跑去庞大的中央仓库深处翻找。
- **后端服务（Service层）= 导购员**：负责接收用户的请求（顾客的需求），他先去临时货架（缓存）看看有没有，有就直接拿给顾客。如果没有，他就去中央仓库（数据库）提货，并顺便放一份到临时货架上方便下次取用。
- **用户请求 = 顾客**。
- **消息队列 = 传送带**：用来将新订单任务传送到打包、发货等不同区域。

极限编程

系统命名隐喻举例：

原来

假设团队正在开发一个**电子商务平台**。

没有系统隐喻时，对话可能是这样的：

程序员A：“我在那个处理用户请求的Servlet里加了点逻辑，调用DAO层之前先检查一下Redis缓存里的KV值，如果命中就直接返回，不然就穿透到MySQL用ORM映射对象，再回写到缓存里。”

产品经理：“...哦。”（完全没听懂，无法参与讨论）

新入职的程序员C：“...我得先看看你的代码是哪个类的。”（理解成本高）

这种沟通充满了技术黑话（Servlet, DAO, Redis, KV, ORM），非技术人员和新人根本无法快速理解。

现在

产品经理：“我觉得‘下单’这个流程太慢了，顾客等得不耐烦。”

程序员A：“是的，我发现是因为我们的‘导购员’（Service代码）每次都要跑去‘中央仓库’（数据库）拿库存数据，即使数据从来没变过。我们可以在他身边放个‘小本本’（本地缓存）或者优化一下‘临时货架’（Redis）的存放策略，把库存信息记下来，这样大部分时候他看一眼小本本就行了。”

新入职的程序员C：“哦，我明白了，那是不是在‘导购员处理流程’（Service方法）里加一个‘检查小本本’（查询本地缓存）的逻辑？”

所有人：“对！就是这个意思！”

选择合适的敏捷方式

并不需要完全拘泥于某一具体敏捷流派，适合自己团队的才是最好的。
Thoughtworks经常为国内的华为、腾讯等公司做敏捷开发培训

Thoughtworks：50%Scrum+40%XP+10%KanBan

软件项目管理

软件项目管理

软件项目管理的特殊性：
1、软件是纯知识产品，其开发进度和质量很难估计和度量，生产效率也难以预测和保证。
2、软件系统的复杂性也导致了开发过程中各种风险的难以预见和控制。

> 例：Windows这样的操作系统有1500万行以上的代码，同时有数千个程序员在进行开发，项目经理都有上百个。这样庞大的系统如果没有很好的管理，其软件质量是难以想象的。

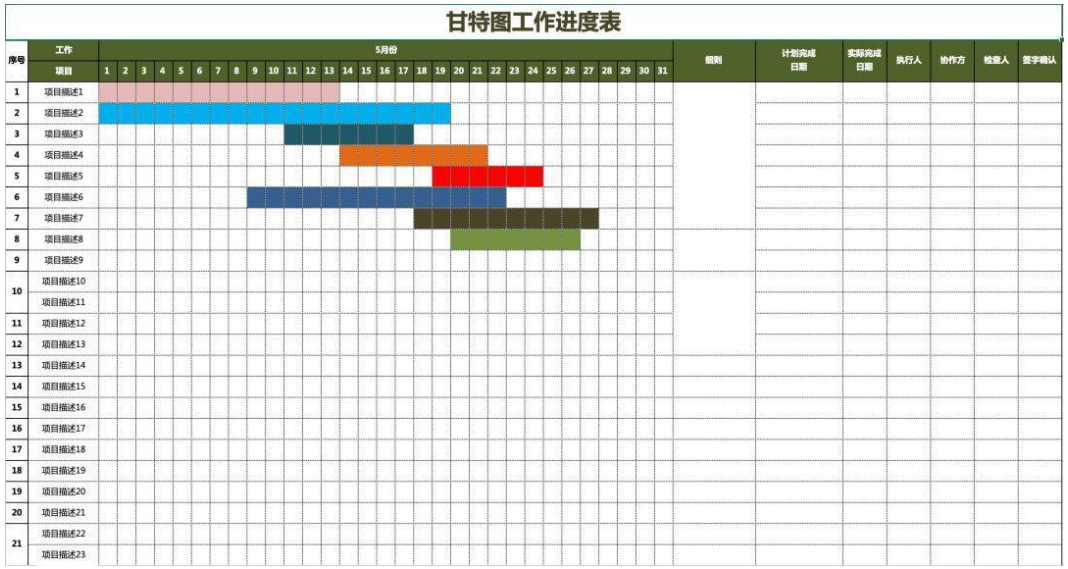
软件项目管理的内容主要包括：人员的组织与管理，软件度量，软件项目计划，软件任务跟踪，软件配置管理等。

强大的工具往往可以帮助我们事半功倍：
1、常用项目管理工具：看板、甘特图、燃尽图
2、常用项目管理软件：Project、Teambition、PingCode

制定和追踪项目计划

甘特图（Gantt chart）：通过条状图来显示项目任务和其他相关要素的内在关系随时间进展的情况，以提出者Henry Laurence Gantt的名字命名。

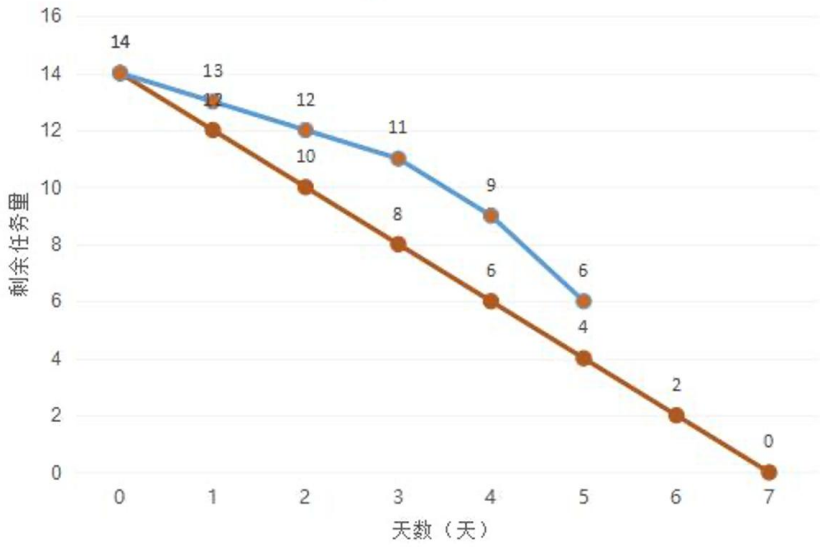
任务：是指软件开发中的一个具体工作。
资源：是指完成任务所需的人员、设备和原材料等。
工期：是完成某项任务所需活动工作时间的总长度，通常是从任务开始日期到完成日期的工作时间量。
里程碑：是一个工期为零，用于标识关键项目节点的重要日程事项。



制定和追踪项目计划

燃尽图（burn down chart）：是用于表示剩余工作量的工作图表，横轴表示时间，纵轴表示剩余工作量，可以直观的预测工作的完成进度，常用于敏捷开发当中。

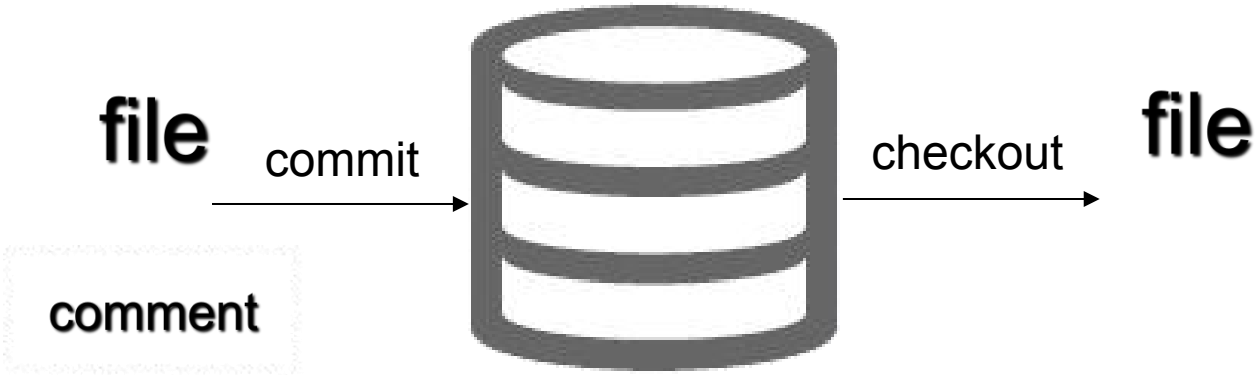
若实际工作线高于理想工作线时，则意味着剩余的工作量比原先预测的多，并且项目落后于计划。
若实际工作线低于理想工作线，则意味着剩余的工作量比原先预测的少，并且项目提前完成。



版本控制与代码协作

版本控制

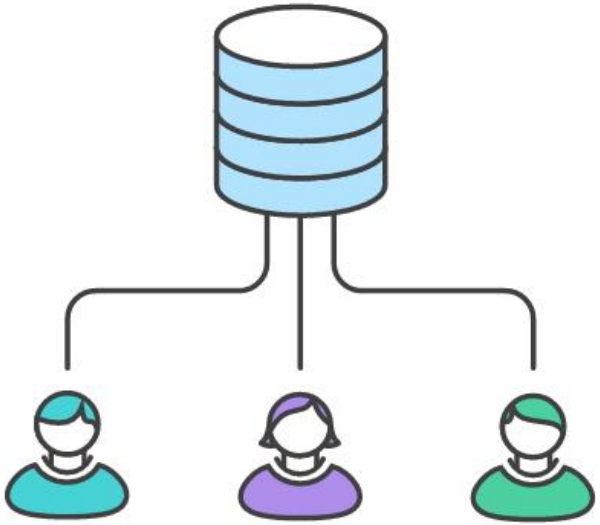
版本控制软件也被称为代码时光机。有了版本库，代码任何更改都会保存起来，并且我们可以对每一次更改提交添加评论（comment），说明更改的内容或目的。



版本库

版本控制

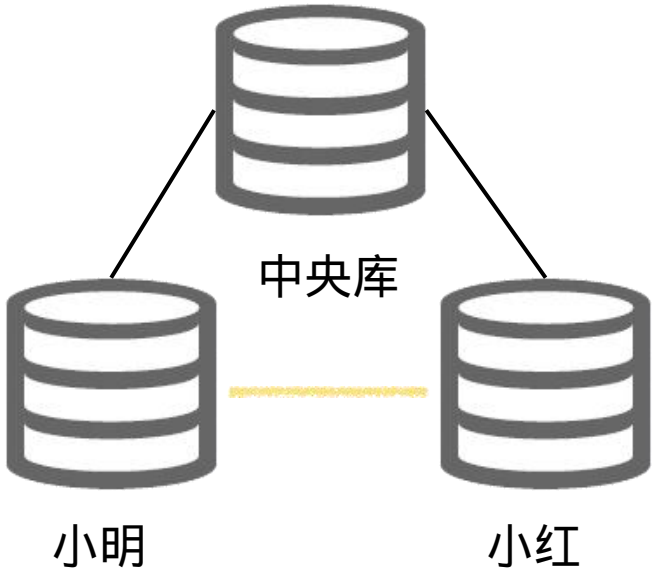
版本控制系统分类：



集中式：如SVN、CVS

做什么事都需要与中央库进行交互，包括查看日志。

缺点：**单点失效**

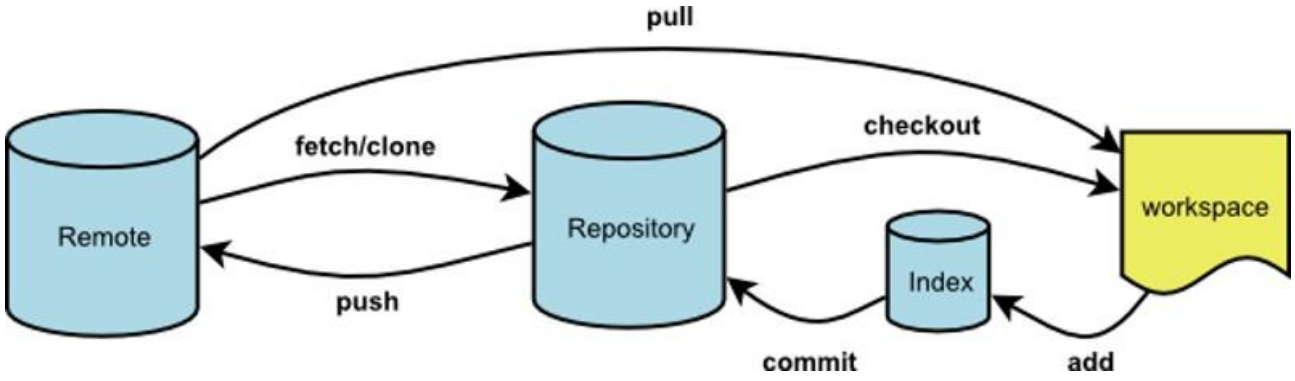


分布式：Git

只需要第一次与中央库进行交互（克隆版本库），之后只需要单机提交到本地版本库。在合适的时候，再与“中央库”进行合并。

版本控制软件——git

git init
git remote add origin git@github.com:帐号名/仓库名.git
git clone git@github.com:帐号名/仓库名.git
git add
git commit -m
git push origin master
git pull origin master
git fetch origin
git checkout
git rm
git branch
git status
git log
……



版本控制基本概念

版本

每一次正确的代码提交，会为版本库生成一个版本，所有版本可以基于依赖关系构成一条代码演进的时间线（树）。

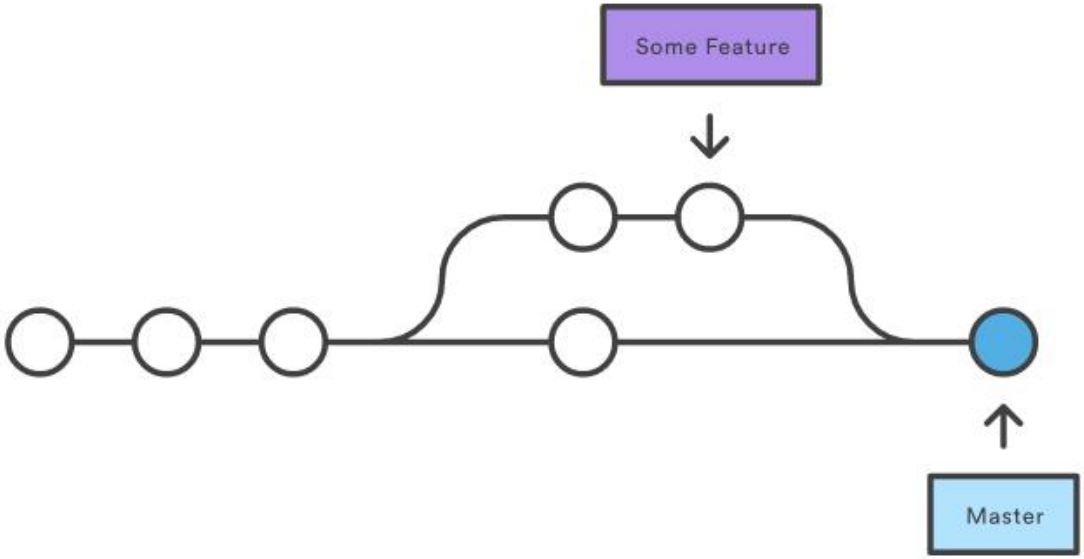
分支

使用分支意味着可以把你的工作从开发主线上分离开来，以免影响开发主线。分支可以根据不同的目的进行划分。

分支分类

1）以功能开发为导向

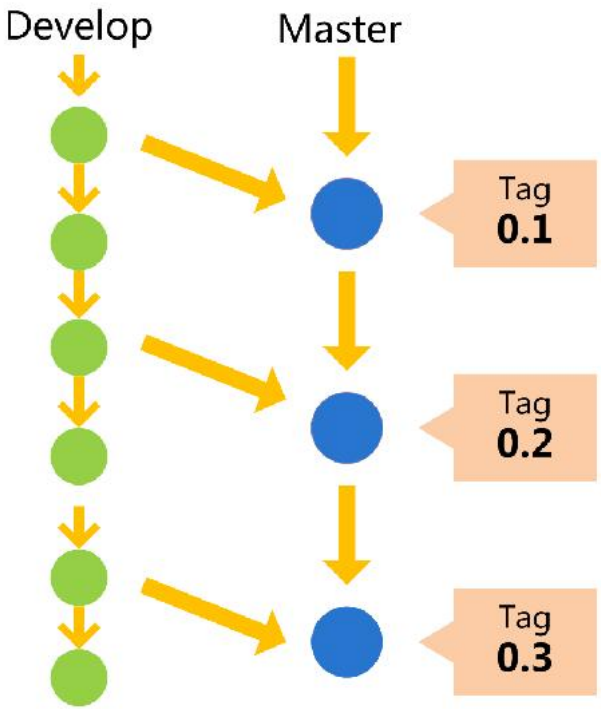
基于功能增加来完成迭代的项目，产生多个分支的目的是为了单独开发某一功能。最终为了实现统一的整体，需要在功能开发完成之后进行分支合并（merge）。



分支分类

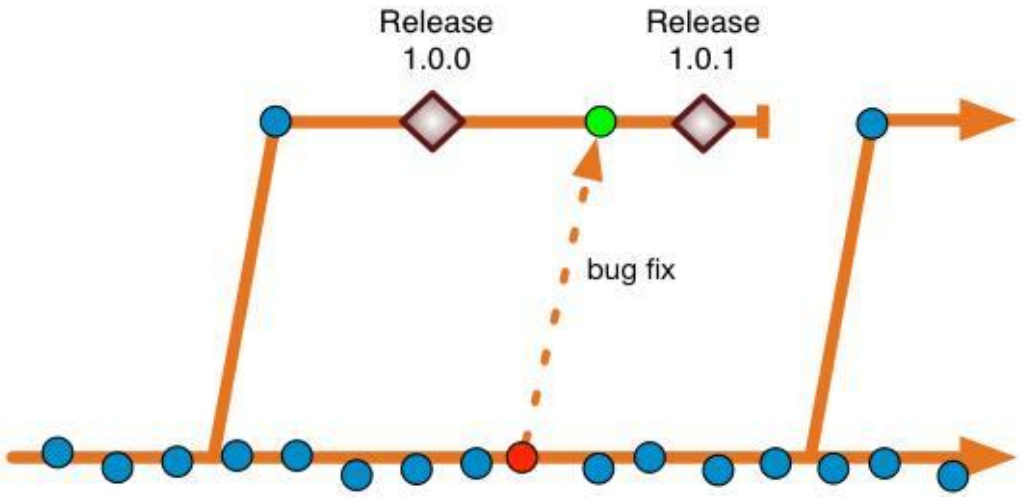
2）以软件状态为导向

我们可以将分支设置为多个：如用于给用户提供稳定的软件版本master主分支；用于项目开发的develop分支；用于软件发布准备的release分支；用于系统维护、修复bug的hotfix分支等。



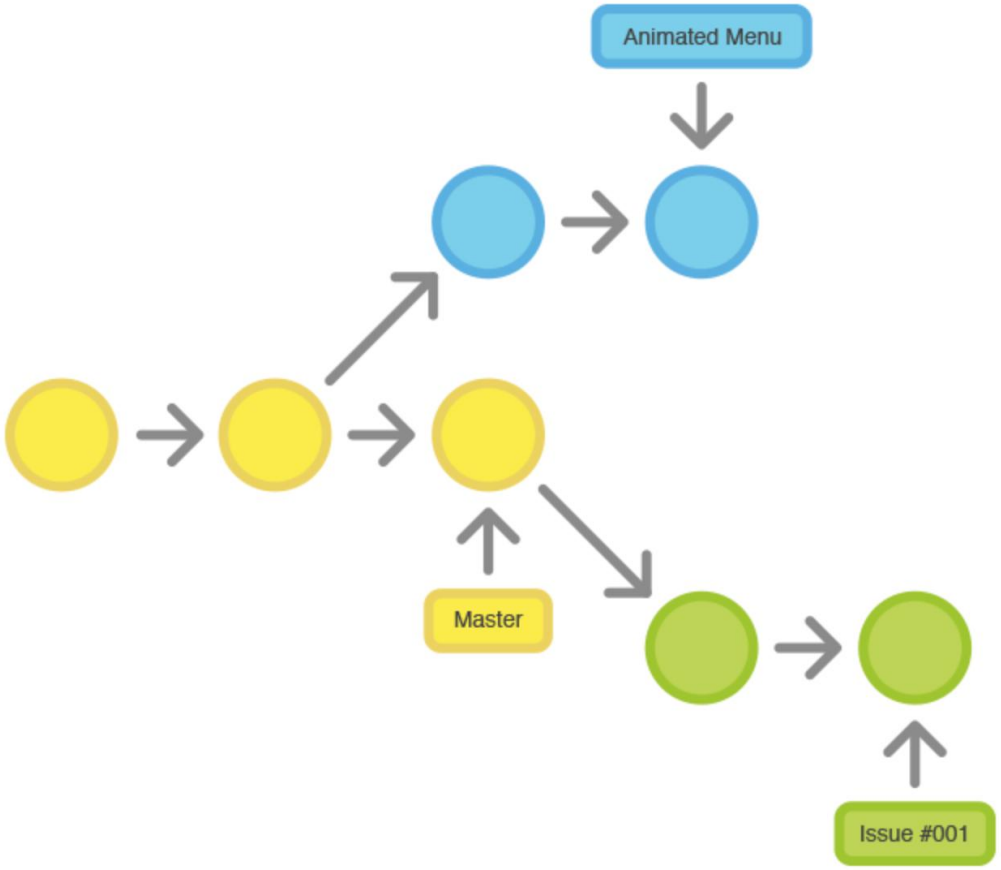
软件协作工作流

1、集中式工作流（trunk工作流）



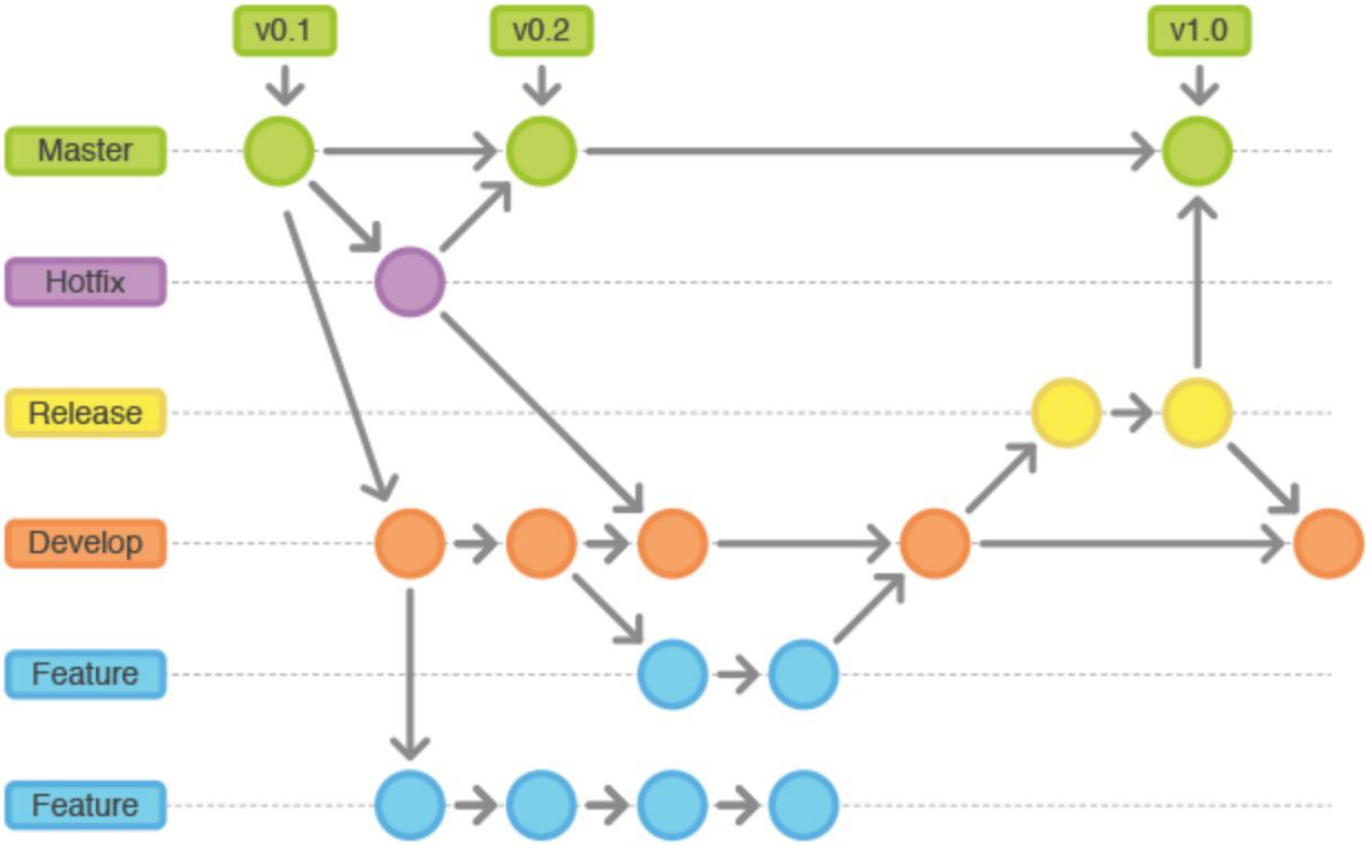
软件协作工作流

2、功能分支工作流



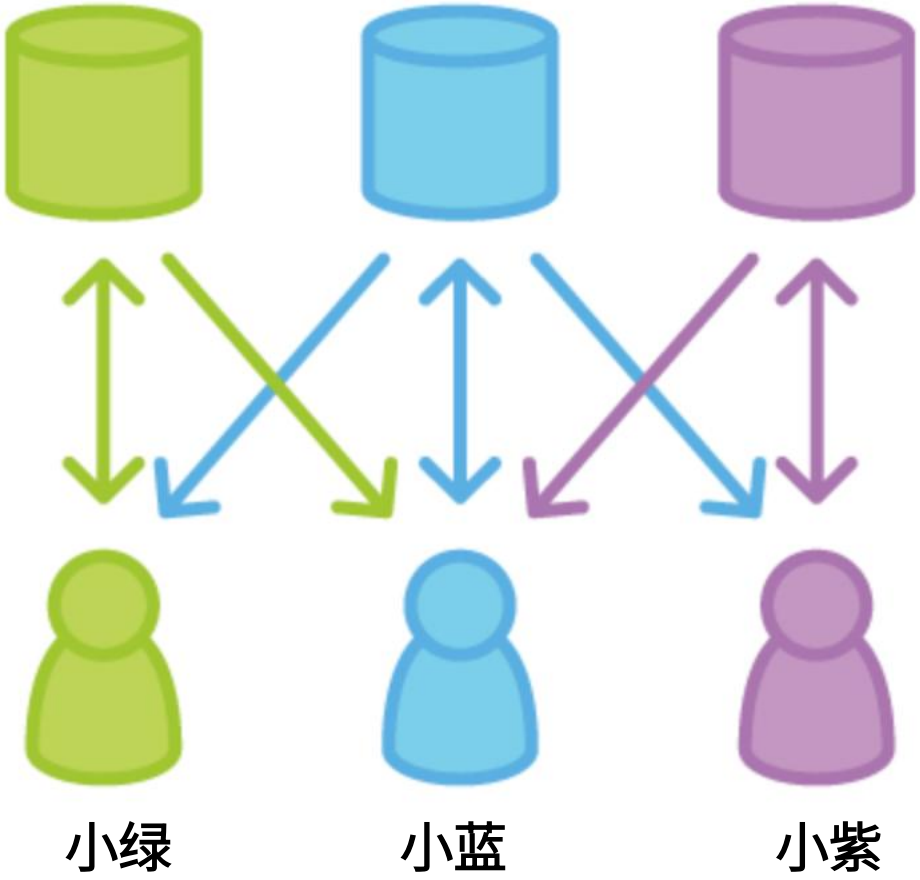
软件协作工作流

3、 gitflow工作流



软件协作工作流

4、forking工作流



实验环节

完成本课程实验三：

- 实验内容见《实验指导书3》
- 要求了解Teambition或类似项目管理软件的用法
- 要求掌握版本管理软件git及远程仓库github/gitee的用法
- 不需要提交实验报告

项目要求：

- （鼓励）使用Teambition或类似软件进行项目协作管理
- 采用Github/gitee创建在线版本库并使用git进行项目开发
- 选定一个合适的软件协作工作流进行协作开发